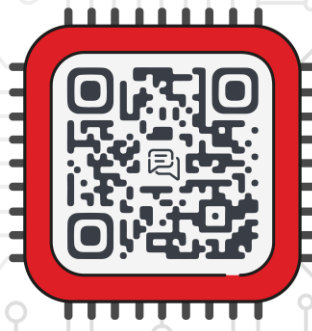




عملي

5

9

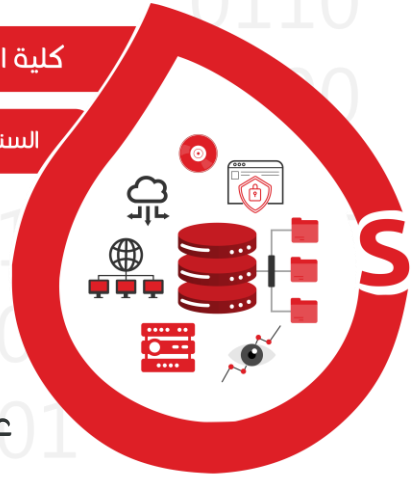
1350 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Index & Filter

عملي مشترك



RB Informatcs; 2025/11/24

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

في المحاضرة السابقة تحدثنا عن نوعي الفهارس في قواعد البيانات:

▪ Clustered Index

▪ Non-Clustered Index

ورأينا أن ال Non-Clustered Index أحياناً يحتاج إلى عملية Lookup لجلب البيانات غير الموجودة ضمن ال Leaf Nodes الخاصة به، مما يزيد من كلفة تنفيذ الاستعلام (Execution Cost).

ولحل هذه المشكلة ظهرت فكرة استخدام:

▪ Included Columns داخل Non-Clustered Index أي إضافة أعمدة إضافية داخل Leaf Node لتجنب

عمليات ال Lookup.

⚡ تذكرة:

ماهي عملية ال Lookup؟

ال Lookup هي عملية يقوم فيها SQL Server بالرجوع إلى الجدول الأصلي (Clustered Index / Heap) لجلب البيانات الناقصة التي لا يحتويها ال Non-Clustered Index.

⚡ لماذا تعتبر عملية Lookup سيئة أو مكلفة؟

- لأنها تحدث لكل صف يطابق الشرط.
- وإذا كان عدد الصفوف كبيراً ← تصبح العملية بطيئة جداً.
- وهذه الكلفة تظهر في مخطط التنفيذ (Execution Plan).

سنقوم ببناء TABLE بسيط:

```
SQLQuery1.sql - not connected*
CREATE TABLE EMP(
EMP_ID INT PRIMARY KEY,
NAME VARCHAR(20) NOT NULL,
GENDER CHAR(1) NOT NULL,
SALARY INT NOT NULL ,
PHONE INT NOT NULL,
ADDRESS Varchar (100),
);
```

ولنقم بالإضافات الآتية:

```
INSERT INTO EMP VALUES (3, 'MAHERE', 'M', 8000, 111, 'ALEPPO');
INSERT INTO EMP VALUES (5, 'SARA', 'F', 5800, 444, 'DAMAS');
INSERT INTO EMP VALUES (1, 'AHMAD', 'M', 9400, 555, 'DAMAS');
INSERT INTO EMP VALUES (4, 'MAHA', 'F', 8500, 222, 'DAMAS');
INSERT INTO EMP VALUES (7, 'FARES', 'M', 7000, 333, 'ALEPPO');
```

عند تنفيذ الاستعلام الآتي:

```
SELECT * FROM emp;
```

	EMP_ID	NAME	GENDER	SALARY	PHONE	ADDRESS
1	1	AHMAD	M	9400	555	DAMAS
2	3	MAHERE	M	8000	111	ALEPPO
3	4	MAHA	F	8500	222	DAMAS
4	5	SARA	F	5800	444	DAMAS
5	7	FARES	M	7000	333	ALEPPO

سنجد بأنه مرتب.

عند تنفيذ Query آخر يعتمد على salary، سيقوم SQL Server أحياناً بـ:

Clustered Index Scan

لأنه لا يستطيع استخدام ال Non-Clustered Index بشكل فعال، وقد يلجأ إلى Lookup لجلب البيانات الناقصة. إضافة Include لل Non-Clustered Index:

```
CREATE NONCLUSTERED INDEX ix_emp
ON emp (salary)
INCLUDE (name);
```

سيتم تخزين كل من salary و name داخل ال Leaf Node.

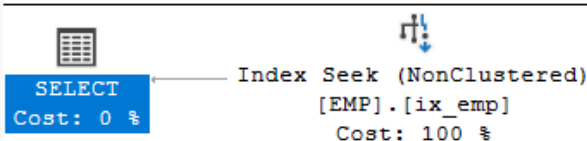
وبالتالي عند تنفيذ استعلام يطلب كلا الحقليين:

```
SELECT NAME
FROM EMP
WHERE SALARY > 7000
```

سنلاحظ في ال Execution Plan أنه يستخدم:

Index Seek (Non-Clustered)

Query 1: Query cost (relative to the batch): 100%
SELECT NAME FROM EMP WHERE SALARY > 7000

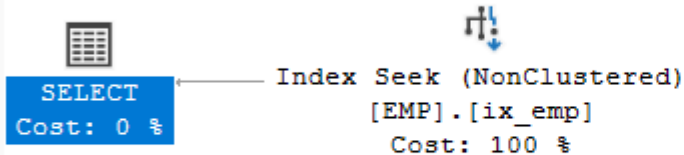


لنقم بعمل الاستعلام الآتي:

```
SELECT NAME
FROM EMP
WHERE SALARY >= 7000
AND NAME LIKE '%S%'
```

نلاحظ انه بال Execution Plan:

Query 1: Query cost (relative to the batch): 100%
 SELECT NAME FROM EMP WHERE SALARY >= 7000 AND NAME LIKE '%S%'

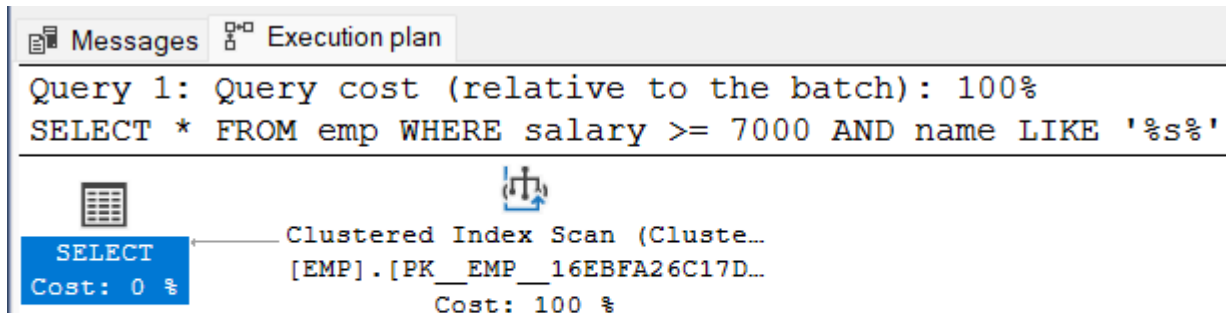


قام بعمل Index Seek لكن في الحقيقة هو قام بعمل ال Index Seek وشيء يدعى PREDICATE.

PREDICATE

في الاستعلام السابق، يتم النظر أولاً إلى شرط WHERE، لأنه غالباً ما يكون أول ما يُفحص عند تنفيذ الاستعلام. في مثالنا، الوصول إلى المفتاح (مثل salary) يتم عبر Index Seek، ثم يُنفذ Scan للبحث عن الأسماء المطلوبة. هذا النوع من الشروط يُعرف باسم Predicate. سنقوم بتنفيذ ال Query التالية:

```
SELECT * FROM emp
WHERE salary >= 7000
AND name LIKE '%s%'
```



نلاحظ أنه في ال Execution Plan قام بعمل scan لأنه نظر إلى قسم ال criteria ووجد أنه يمكن الحصول إليه من خلال non-clustered index، ولكن للحصول على قسم ال selection سيتوجب عليه العودة إلى الجدول كاملاً والحصول عليه (lookup)، فاختار الكلفة الأقل وهي clustered index scan.

النتيجة:

- يقوم ال optimizer بحساب الكلفة (سيختار ال execution plan المناسب).
- يجب علينا أن نختار ال key بعناية، في المثال السابق إذا وضعنا مثلاً name + salary، لا يوجد لدينا أي أسماء متشابهة في ال Data، سنحصل على index فيه Records تساوي ال Data الأصلية، أي لم نستفد شيء من ال Index.

Filter Index

ال Filtered Index هو فهرس يتم بناؤه على جزء من البيانات بناءً على شرط محدد.

مثال:

```
SELECT EMP_ID, NAME
FROM emp
WHERE phone IS NULL;
```

وقد قمنا ببناء نوعين من الفهارس في هذا الجدول وهي:

■ CLUSTERED INDEX (PK)

■ NON CLUSTERED INDEX (ON SALARY, EMP NAME)

عند تنفيذ هذا الاستعلام، يتم عمل Clustered Scan لأن ال Non-Clustered الموجود مسبقاً غير مفيد لعدم توافق ال Key مع شرط WHERE.

أي أنه قام بـ:

عمل Clustered Scan لكي يستطيع تنفيذ الاستعلام واختبر الشرط بال PREDICATE فجلب جميع الأسطر المحققة للشرط (Phone IS NULL).

ومن هنا أتت فكرة ال Filtered Index:

```
CREATE NONCLUSTERED INDEX ix_tbemp1
ON dbo.emp (Name, phone)
WHERE phone IS NULL;
```

وهنا سيحتوي الفهرس فقط على السجلات التي فيها phone IS NULL.

النجاح يبدأ عندما تخرج من منطقة الراحة

نتيجة هذا الفهرس:

- الفهرس أصغر حجماً.
 - أسرع في البحث.
 - لا يحتوي إلا على قيم ال phone = NULL.
 - أي سجل غير NULL لن يكون ضمن الفهرس.
- أي أنه قام ب:

تخزين ال Non Clustered Index في ال leaf node وال NAME وال phone حيث أنه يحتوي قيم ال null حصراً وأي سجل لا يحوي على قيم ال null في عامود ال phone لن يتم تخزينه في ال leaf node.

وبهذه الطريقة أصبح لدينا الفهرس لا يمتلك مؤشر بالضرورة على كامل السجلات وإنما السجلات المحققة للشرط حصراً.

ماذا لو أعدنا الاستعلام بعد بناء ال Filter Index؟

```
Select EMP_ID, NAME
From dbo.emp
Where phone is null
```

- نلاحظ أنه الشرط هنا هو نفسه ال filter الخاص بالفهرس الذي تم بناؤه ولكن ال selection غير موجودة بشكل كامل.
- index يحمل اسم الموظف ورقم الهاتف.
- الاستعلام يطلب id الموظف واسمه.
- لذلك يتم تنفيذ بناء على clustered index scan.

لماذا لم يقم بتنفيذ Seek؟

حتى يتم استخدام الفهرس المفتر، يجب أن يتوافق:

1. ال Key.
2. الشرط Criteria (WHERE).
3. الحقول المطلوبة.

فمثلاً إذا كان الاستعلام يطلب EMP_ID وهو ليس موجوداً ضمن الفهرس، سيضطر SQL Server للعودة للجدول الأساسي، فينفذ:

- Clustered Index Scan.

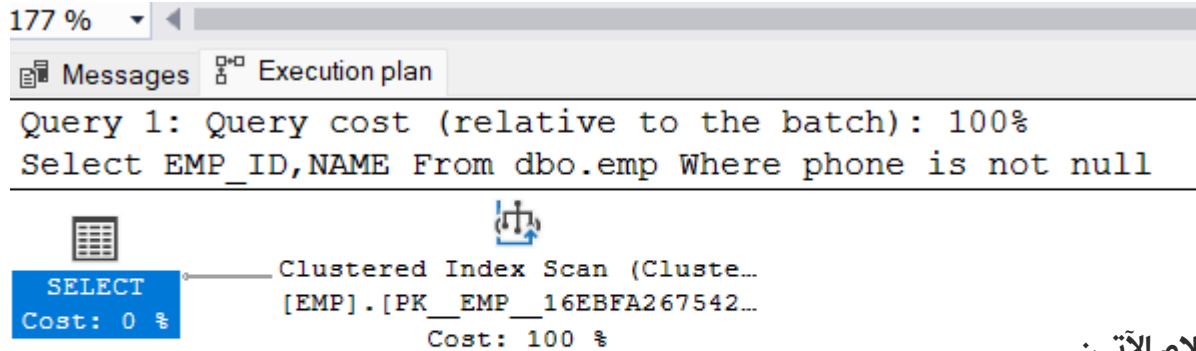
لا تقارن بدايتك بنهاية غيرك، فلكل طريقته وزمنه

```

Select EMP_ID, NAME
From dbo.emp
Where phone is not null

```

هنا نحن نطلب السجلات التي لا يخزنها أساساً في ال filter index الذي قمنا ب بناءه ولا يملك مؤشر عليها وفي هذه الحالة سيتم عمل ال clustered index.



لدينا الاستعلام الآتي:

```

SELECT EMP_ID, NAME
FROM dbo.emp
WHERE phone IS NOT NULL;

```

والفهرس المفلتر مبني على:

Where phone IS NULL

إذا أجبرنا ال Optimizer على استخدام ال Filtered Index باستخدام Hint مثل:

```

SELECT EMP_ID, NAME
FROM dbo.emp WITH (INDEX(ix_ttemp1))
WHERE phone IS NOT NULL;

```

سيظهر Error ولن ينفذ الاستعلام:

لماذا؟

لأن ال Filter Index لا يحتوي على أي صف من الصفوف التي قيمتها NOT NULL، لأنه مبني حصراً على: phone IS NULL

وبالتالي:

- البيانات التي نريدها غير موجودة داخل الفهرس.
- محاولة إجبار النظام على استخدام فهرس لا يحتوي البيانات المطلوبة تؤدي إلى خطأ (Error).

تعَب اليوم هو راحة الغد، فلا تستهن بقيمة الجهد

ال Filter Index يحتوي فقط على الحقول التالية داخل ال Leaf Nodes.



▪ .Name

▪ .phone

بينما الاستعلام يطلب:

▪ .EMP_ID

▪ .Name

وهنا تظهر المشكلة...

استخدامات Filter Index

1. تقليل كلفة البحث، خاصة عندما يحتوي الجدول عدداً كبيراً من قيم NULL.
 2. يسمح بإنشاء ال unique index على الحقول التي تحتوي NULL بشكل لا يسبب أخطاء.
- سنقوم بإنشاء ال unique index ثم عمل إضافات بقيم فريدة في حقل ال phone.

```
CREATE UNIQUE INDEX IX_EMP
ON DBO.EMP(PHONE)
```

```
INSERT INTO EMP VALUES (3, 'MAHERE', 'M', 8000, 111, 'ALEPPO');
INSERT INTO EMP VALUES (5, 'SARA', 'F', 5800, 444, 'DAMAS');
INSERT INTO EMP VALUES (1, 'AHMAD', 'M', 9400, 555, 'DAMAS');
INSERT INTO EMP VALUES (4, 'MAHA', 'F', 8500, 222, 'DAMAS');
INSERT INTO EMP VALUES (7, 'FARES', 'M', 7000, 333, 'ALEPPO');
```

نلاحظ بأنه ال UNIQUE يسمح بإنشاء قيمة null لمرة واحدة فلو حاولنا القيام بعملية إضافة لقيمتي null في الحقل سيظهر لدينا خطأ لأنه يعتبر ال null كقيمة مكررة.

```
INSERT INTO dbo.emp VALUES (8, 'mm', 'M', 7000, null, 'ALEPPO')
INSERT INTO dbo.emp VALUES (9, 'bb', 'M', 7000, null, 'damas')
```

195 %

Messages

Msg 2601, Level 14, State 1, Line 81

Cannot insert duplicate key row in object 'dbo.EMP' with unique index 'IX_EMP'. The duplicate key value is (<N>). The statement has been terminated.

Completion time: 2025-12-13T21:07:00.8199973+03:00

الحل؟

نقوم بعمل drop لل unique index.

```
Drop index emp.IX_EMP
```

والآن بعد حذف ال Unique Index القديم، نقوم بإنشاء Unique Index مفلتر باستخدام

WHERE phone IS NOT NULL

```

Create unique index IX_EMP
On
Dbo.emp (phone)
Where phone is not null

```

بحيث تطبق خاصية الفريدة (uniqueness) فقط على السجلات التي تحتوي رقم هاتف بينما يُسمح بتكرار القيمة NULL في حقل phone دون حدوث أي خطأ.

ملاحظة:

لا يمكننا تكرار القيم التي ليست null:

```
INSERT INTO dbo.emp VALUES (7,'mm','M',7000,null,'ALEPPO')
```

```
INSERT INTO dbo.emp VALUES (9,'bb','M',7000,null,'damas')
```

```
INSERT INTO dbo.emp VALUES (11,'cc','M',7900,111,'damas')
```

وهنا نضمن بأنه:

لن يتم تكرار أرقام الموظفين unique

وبنفس الوقت نسمح بتكرار القيمة null في حقل أرقام الموظفين.

Covered Index

المقصود ب Covered Index هو بناء index يغطي كل الاستعلامات (query) التي تحصل على ال Table، وفي الواقع لا وجود ل index قادر على تقديم كامل الاستعلامات (باستثناء index وحيد وهو ال Clustered Index الافتراضي عن طريق ال PK).

ولكن هل يمكننا بناء Non-Clustered Index على جدول بحيث يكون Covered على أي استعلام يتم طلبه من الجدول؟

الجواب:

لا، لأن ال criteria قد تختلف من query لأخرى، وأي اختلاف بال criteria يستدعي اختلاف بال key index، ومن هذا المنطلق لا يمكن ل Non-Clustered Index أن يغطي (Covered) كامل الاستعلامات.

دائماً حتى يتم تنفيذ ال index يجب أن تكون ال criteria في الاستعلام حتماً مطابقة لأول key index value، لذلك مهما قمنا بتكبير ال key index وأضفنا له value. ما لم توافق ال criteria أول key index، لن تستفيد من ال index.

مثال للتوضيح: لدينا ال Index التالي

```

CREATE NONCLUSTERED INDEX example
ON emp (Firstname, Lastname, gender)

```

ولدينا الاستعلام التالي:

```
SELECT * FROM emp
WHERE gender = 2;
```

هل الاستعلام سيستفيد من ال index؟

- لا، لأن أول مفتاح (key) في ال index غير موجود في شرط الاستعلام (criteria).
 - أي استعلام لا يحتوي شرطه على أول مفتاح في ال index لن يستفيد من هذا ال index.
- ومن هنا يمكننا الاستنتاج بأنه عند بناء الفهرس نقوم بالبحث عن أكثر حقل تكرر في الشرط ونجعله كأول key index value

سؤال؟

أيهما أفضل:

- بناء Non-Clustered Index واحد يحتوي على Include لكل الحقول؟
- أم بناء عدة Non-Clustered Indexes بنفس المفتاح ولكن كل واحد يحتوي على جزء مختلف من الحقول في ال Include؟

مثال:

إذا كانت 80% من الاستعلامات تحتوي على شرط لل Date
هل من الأفضل بناء Index واحد بمفتاح Date ويحتوي على Include لكل الحقول؟
أم بناء 5 Indexes بنفس المفتاح Date وكل واحد يحتوي على جزء مختلف من الحقول؟

الجواب:

- إذا بنينا Index واحد يحتوي على 32 حقل في ال Include (وهو الحد الأقصى)، فسيصبح حجمه قريب من حجم الجدول الأساسي، مما يجعله مرهقاً في الأداء.
- الأفضل: بناء عدة Indexes بنفس المفتاح، وكل واحد يحتوي على مجموعة جزئية من الحقول في ال Include، مما يجعل الحجم مقبولاً والأداء أفضل.

الملخص:

- لا يوجد Index واحد يغطي 100% من الاستعلامات.
- ال Covered Index هو مفهوم نظري، ويمكن بناء أكثر من Index التغطية الاستعلامات.
- في النهاية، سيعتمد التنفيذ على ال Clustered Index في مرحلة ما.
- يمكن أن يغطي استعلام واحد أكثر من Index.

The End

النجاح لا يأتي صدفة

بل هو نتيجة صبر وتعب واستمرار دون استسلام